GUIDE 09 · QUERIES

Export a large result without running out of memory

The latest version is at anantharajuc.github.io/QueryAPIGate-website/Guide.dc.html?g=09-stream-a-large-export. This copy was made on 4 November 2026.

Time: 10 minutes. **You'll end up with:** a real, measured understanding of what `?stream=true` actually buys you - not taken on faith. Every number below is from a real run against a real MySQL container with 500,000 and 1,000,000 real rows, memory sampled from the actual server process.

The problem this solves

Every normal QueryAPIGate response is paginated (`?page` / `?page_size`), capped by `QUERYAPIGATE_MAX_PAGE_SIZE` (default 1000) - so an ordinary call is already memory-safe. The problem `?stream=true` solves is different: **getting the whole result in one response**, without paging through it in a loop yourself, for an export where you genuinely want every row.

Proof: the same 500,000 rows, two ways, measured

A full-table export of 500,000 rows, **without** streaming (`?page_size=500000`, with `QUERYAPIGATE_MAX_PAGE_SIZE` raised to allow it) - watching the actual server process's memory (`VmRSS`) while it ran:

```shell
curl 'http://127.0.0.1:5000/execute_sql?page_size=500000' -H 'X-API-Key: demo-key' -H 'Content-Type: application/json' \
  -X POST -d '{"sql": "SELECT * FROM events", "connection_name": "bigdata"}'
```

Peak memory: ~374-397 MB (baseline was ~42 MB) for a 74 MB response - building the full list of rows and JSON-encoding all of them at once costs several times the size of the final payload.

The exact same data, streamed instead:

```shell
curl 'http://127.0.0.1:5000/execute_sql?stream=true&format=csv' -H 'X-API-Key: demo-key' -H 'Content-Type: application/json' \
  -X POST -d '{"sql": "SELECT * FROM events", "connection_name": "bigdata"}' -o export.csv
```

Peak memory: ~60 MB. Same 500,000 rows, same query, a sixth of the memory.

The actual "constant memory" claim, verified by doubling the data: the same streamed export against 1,000,000 rows (twice the data) peaked at **~60 MB again** - not twice. That's the real guarantee: memory stays flat as the result grows, because rows are read from the database and written to the response as they come off the cursor, never held all at once.

What makes a query eligible

- **Only `csv`, `tsv`, `ndjson` and `parquet`** - verified: `?stream=true&format=json` is rejected outright (`"stream=true only supports these formats: csv, ndjson, parquet, tsv"`). There's no way to stream `json` / `xml` / `yaml` / `xlsx` - each needs the whole structure before it can be valid, which defeats the point. Parquet is written by DuckDB to a temporary file first and then sent - verified on the example APIs' 20,000-row export: 513 KB of Parquet against 2.2 MB of CSV.
- **No `page` / `page_size`** - verified: combining them is rejected (`"stream=true exports the whole result and does not accept page/page_size"`). Streaming exports the *whole* result; pagination and streaming are two different answers to "how much do I get," not combinable.
- **Always read-only, unconditionally** - verified directly: with `QUERYAPIGATE_ALLOW_WRITES=1` set server-wide, the exact same `DELETE` statement succeeds normally but is rejected outright with `?stream=true` (the same generic "only read-only statements are allowed" message, which is a little misleading here - it reads as if setting `QUERYAPIGATE_ALLOW_WRITES` would fix it, but it won't; a streamed query is read-only no matter what). A large export has no business mutating data, so this isn't configurable.
- **Not available for Mongo yet** - see Connect to MongoDB.

Works the same way on a saved query (`GET /q/<name>?stream=true&format=csv`) as it does ad hoc (`POST /execute_sql`) - same flags, same restrictions, same code path.

Which dialects actually stream from the database itself

Not every dialect is equal here, and this project is explicit about which is which rather than implying uniform behavior:

DIALECT	WHAT ACTUALLY HAPPENS
MySQL	A real unbuffered cursor - rows come from the database one at a time, genuinely constant memory (this guide's own measurement).
PostgreSQL	A named/server-side cursor - same guarantee.
ClickHouse	`execute_iter` - same guarantee.
SQLite, H2, generic `jdbc`, DuckDB	This project's own memory is still capped per batch (1000 rows at a time) - but the underlying engine or driver may buffer more internally than that. SQLite is a local file with no separate driver-level buffering concern; H2/`jdbc` can't have their fetch size tuned through `jaydebeapi`; DuckDB computes its whole result during `execute()` before the first row is even available.

If true constant-memory streaming at real scale matters for your use case, make sure the connection is one of the first three - verified here with MySQL, documented the same way for Postgres/ClickHouse in documentation/DATABASE_CONNECTION_CONFIGURATION.md.

A streamed export holds its connection for the whole download

Worth knowing before sizing your connection pool: a streaming request checks out a pooled connection like any other, but doesn't release it until the client has received the *entire* result - which, for a slow client or a genuinely large export, can be much longer than a typical request. A few large concurrent exports can make a small `QUERYAPIGATE_POOL_SIZE` feel undersized for everything else happening at the same time. Size the pool with that in mind, or set `QUERYAPIGATE_POOL_SIZE=0` to have every request (streamed or not) open and close its own connection instead, if that trade-off suits your deployment better.

Next steps

- Run a scheduled export - `queryapigate export` runs the same streaming path, no server or API key needed, good for a cron job.
- Get the exact CLI command for a saved query.
- Connect a database - pick MySQL, PostgreSQL or ClickHouse if real constant-memory streaming at scale is a requirement, not just a nice-to-have.